

Distributed Search Engine

Project Proposal

Submitted to: Dr. Jeelani

Thapar Institute of Engineering and Technology

Department of Computer Science and Engineering

Hitesh, CSED

Agambir Singh, CSED

Nityam Mantri, CSED

August 2026

1 Higher-order goal

This project aims to design and develop a **scalable distributed search engine** capable of efficiently indexing, processing, and retrieving information from large document collections. The primary objective is to build a production-oriented search platform that overcomes the limitations of traditional single-node search systems through **distributed architecture, efficient indexing techniques, and relevance-based ranking mechanisms**.

The system focuses on applying modern backend engineering principles, **microservices architecture**, and distributed computing techniques. The project emphasizes practical implementation of concepts such as service communication, concurrency management, caching, fault tolerance, and containerized deployment.

2 Problem Statement

Modern applications generate massive volumes of structured and unstructured textual data that require efficient retrieval mechanisms. Traditional single-server search systems face limitations when handling large datasets and increasing user traffic.

A scalable search engine requires optimized indexing structures, parallel query processing, and intelligent ranking methods to provide fast and accurate results.

Common challenges include:

- **Limited scalability:** A single search server becomes a performance bottleneck as the document collection and number of concurrent users increase. The system requires horizontal scaling through distributed search nodes.
- **High query latency:** Sequential searching over large collections results in increased response time. Efficient indexing and parallel query execution are required to achieve low latency retrieval.
- **Poor fault tolerance:** Failure of a single component can impact the complete search workflow. Distributed services improve reliability by isolating failures.
- **Limited search relevance:** Traditional keyword matching may fail to understand semantic relationships between queries and documents. Combining keyword-based ranking with semantic similarity improves retrieval quality.

3 Proposed Solution

3.1 Overview

The proposed system is a **distributed search engine**. The architecture separates different responsibilities such as document ingestion, indexing, query processing, ranking, and storage into independent

services.

The system distributes search operations across multiple **search shards**, where each shard maintains a portion of the indexed data. A central coordinator manages incoming queries, communicates with shards, and combines results to generate the final ranked response.

The major components include:

- **Document ingestion service:** Responsible for collecting documents, performing preprocessing operations, and preparing data for indexing.
- **Indexing service:** Generates and maintains **inverted indexes** that allow efficient document retrieval.
- **Query coordinator:** Handles incoming search requests and manages communication between users and search services.
- **Search shard services:** Maintain distributed portions of the index and execute queries in parallel.
- **Ranking module:** Improves result ordering using relevance scoring techniques such as **BM25 ranking** and semantic similarity.
- **Caching layer:** Reduces repeated query processing time and improves system responsiveness.

3.2 Core workflow

1. Documents are collected through the ingestion pipeline and processed using text normalization and tokenization techniques.
2. The indexing service creates an **inverted index structure** that maps terms to relevant documents.
3. Index data is distributed among multiple search shards to enable parallel retrieval.
4. User queries are received by the coordinator service.
5. The coordinator performs concurrent searches across multiple shards.
6. Results from different shards are merged using a **Top-K ranking algorithm**.
7. The final ranked documents are returned to the user.

3.3 Data Flow Diagram

Figure 1 presents the system’s data flow across three levels of decomposition: Level 0 shows the system’s context with external entities (User and Content Source); Level 1 details the query-handling flow through the coordinator, cache, and shards; and Level 2 breaks down the document indexing pipeline and the per-shard scoring process, including the hybrid BM25 and vector embedding merge.

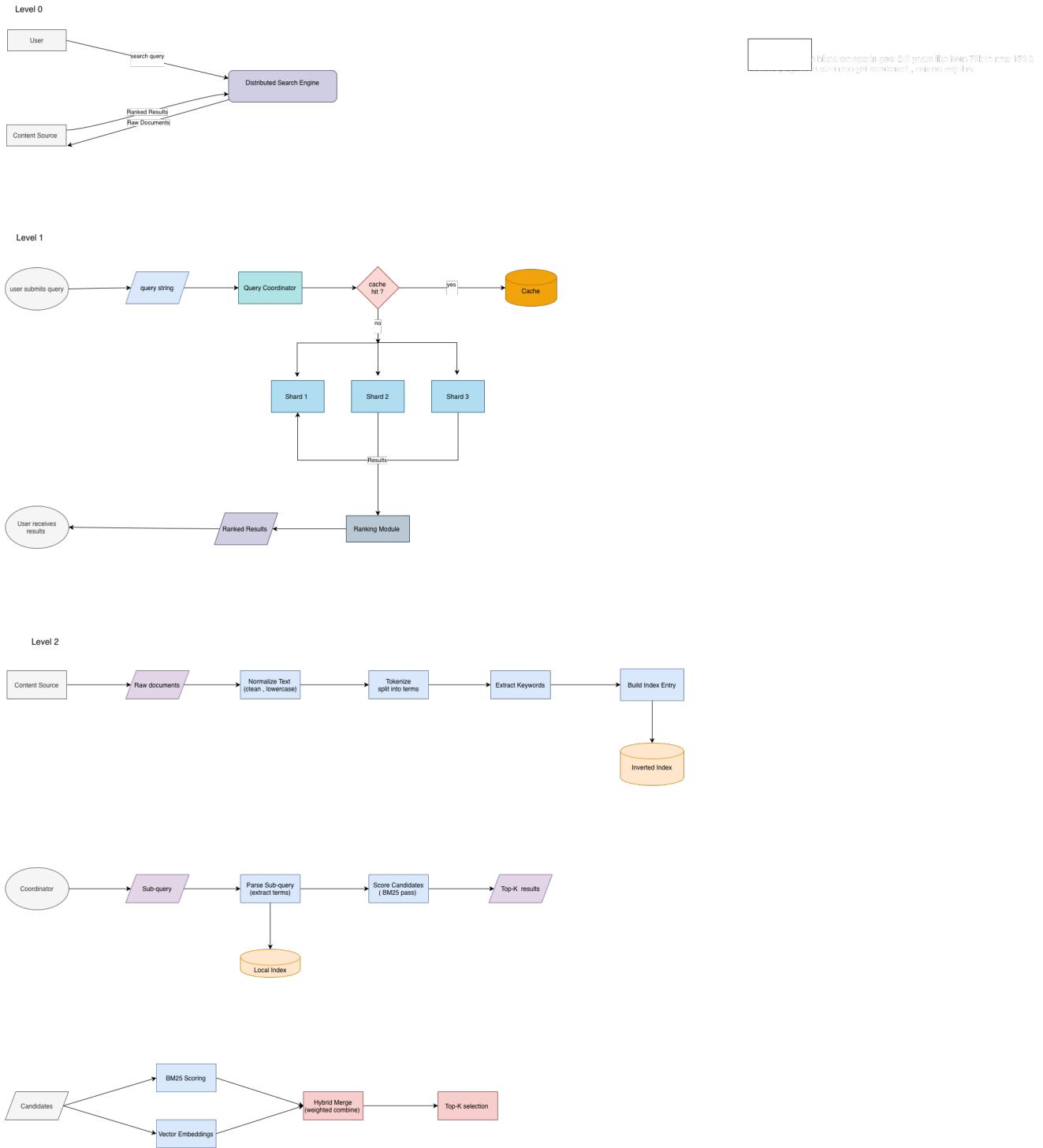


Figure 1: Data Flow Diagram (Levels 0, 1, and 2) for the Distributed Search Engine.

4 Solution Approach

This project focuses on building a **scalable backend system** using distributed system principles rather than developing new search algorithms. The objective is to integrate existing information retrieval methods into a reliable software architecture.

4.1 Indexing and retrieval

The search engine uses an **inverted index** as the primary data structure for efficient document retrieval.

The indexing pipeline includes:

- **Text normalization and tokenization:** Converts raw documents into searchable representations.
- **Keyword extraction:** Identifies important terms required for efficient indexing.
- **Index generation:** Creates document-term mappings for fast retrieval.
- **Efficient storage:** Maintains indexed information for quick access during search operations.

The ranking system combines traditional information retrieval methods with semantic search techniques:

- **BM25 based scoring:** Calculates keyword relevance between queries and documents.
- **Vector embedding similarity:** Captures semantic relationships between queries and documents.
- **Hybrid ranking:** Combines multiple ranking signals to improve search accuracy.

4.2 Distributed architecture

The system follows a **microservice-based architecture** consisting of:

- **Search Coordinator:** Receives queries, distributes requests, and performs final result aggregation.
- **Search Shards:** Maintain independent portions of the search index and process queries concurrently.
- **Indexer Service:** Creates, updates, and manages search indexes.
- **Storage Layer:** Stores documents, metadata, and persistent application information.

4.3 Use Case Diagram

Figure 2 illustrates the primary use cases for the system, showing interactions between the **User** and **System Admin** actors and the core platform functionality, including the include-relationships between document search, shard execution, and hybrid ranking.

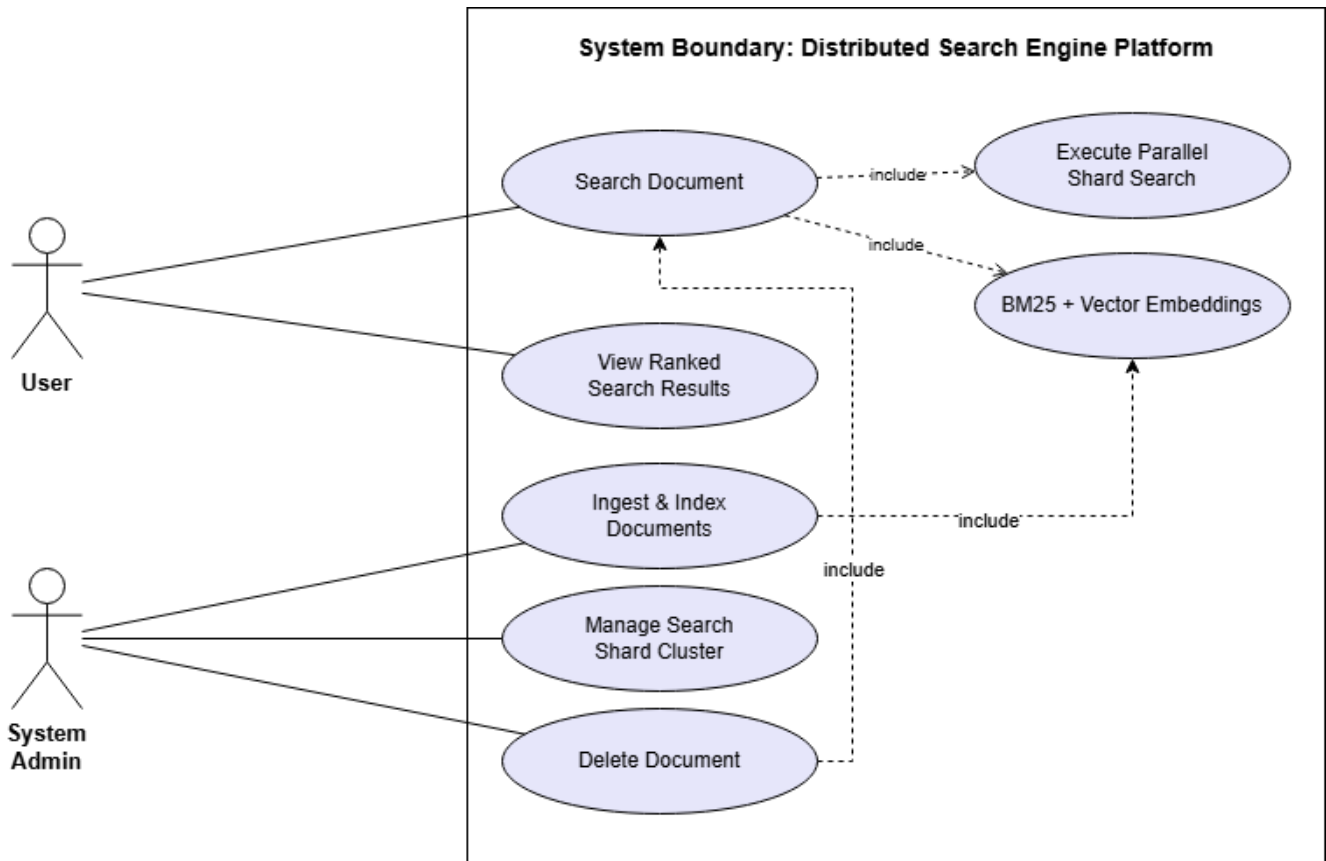


Figure 2: Use Case Diagram for the Distributed Search Engine Platform.

4.4 Service communication and coordination

The system will use:

- **REST APIs** for external user communication.
- **gRPC communication** for efficient internal service communication.
- **Service discovery** for dynamically locating available components.
- **Docker containers** for deployment and service isolation.

5 Evaluation Criterion

The system will be evaluated using measurable performance, scalability, and search quality metrics.

5.1 Primary evaluation metrics

Query latency:

Measures the time required between receiving a query and returning ranked results.

- Target: Maintain low response time under increasing document size and query load.

Search relevance:

Measures the quality of retrieved documents based on ranking accuracy.

- Evaluated by comparing returned results with expected relevant documents.

5.2 Secondary evaluation metrics

- **Indexing throughput:** Measures the rate at which documents can be processed and indexed.
- **Queries processed per second:** Evaluates system handling capacity.
- **Cache hit ratio:** Measures the effectiveness of the caching layer.
- **System availability:** Evaluates reliability during operation.
- **Resource utilization:** Measures CPU and memory usage under concurrent workloads.

6 Scalability

The system is designed to support increasing document collections and user traffic through distributed processing.

Scalability will be achieved through:

- **Horizontal scaling** of search shard services.
- **Stateless backend services** allowing independent deployment.
- **Distributed indexing** for handling large document collections.
- **Database optimization** using indexing and efficient queries.
- **Containerized deployment** for flexible infrastructure management.
- **Parallel query execution** for reducing response time.

7 Engine availability heuristic

The project will use mature software components to focus on system design, implementation, and scalability.

The technology stack includes:

- **Java Spring Boot** for backend microservices.
- **PostgreSQL** for persistent storage.
- **Redis** for caching frequently accessed queries.
- **gRPC** for service-to-service communication.
- **Docker** for deployment and service management.
- **Monitoring tools** for observing system performance.

8 Project scope and deliverables

8.1 Initial deliverable

- Document ingestion pipeline.
- Search API implemented using Spring Boot.
- Basic inverted index implementation.
- Query processing service.
- Ranking module.
- Docker based local deployment.

8.2 Subsequent deliverables

- Distributed shard-based search architecture.
- Parallel query execution.
- Redis caching layer.
- Semantic search using embeddings.
- Monitoring and performance analysis dashboard.

9 Risks and mitigations

- **Scalability issues:** Addressed through distributed architecture and horizontal scaling.
- **Search quality limitations:** Improved through hybrid ranking combining keyword and semantic methods.
- **Service failures:** Managed through monitoring and fault-tolerant communication.
- **Performance bottlenecks:** Reduced using caching and optimized indexing strategies.

10 Summary

This project proposes a **distributed search engine implemented using Java Spring Boot and modern backend technologies**. The system combines efficient indexing, distributed query processing, caching, and relevance-based ranking to provide fast and accurate search results.

The project demonstrates practical distributed systems engineering concepts including **microservices, concurrency, scalability, fault tolerance, and reliable deployment practices**. The final system aims to provide a strong foundation for building production-level search infrastructure.